AGENT INSTRUCTION: E-ITS ISMS Architecture & Business Process (Äriprotsess) v2 API Implementation

1. Context & Core Principles
You are implementing the core domain logic for an E-ITS based Information Security Management System (ISMS - *infoturbe halduse süsteem*)[cite: 3, 5]. E-ITS is fundamentally business-process-centric (*äriprotsessikeskne*). Security always flows from the business needs down to the technical assets (*sihtobjektid*)[cite: 5].

You must strictly enforce the rules set by the Cybersecurity Act (KüTS - *küberturvalisuse seadus*) and the Estonian Information Security Standard (E-ITS) using the **API v2** architecture[cite: 3, 5].

---

2. E-ITS Database Schema Enhancements (v2 Migration)
The existing database schema contains `business_processes`, `damage_assessments`, and `protection_need_summaries`. You must add the following tables and relationships to handle process-to-process interdependencies (*protsesside omavahelised seosed*) and cascade compliance tracking[cite: 5].

A. SQL Migration Script
Apply this schema extension to handle process dependencies without breaking existing core structures:

```sql
-- Create process-to-process dependency tracking table
CREATE TABLE public.business_process_dependencies (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    primary_process_id uuid NOT NULL,    -- The process that depends on another (sõltuv äriprotsess)
    depends_on_process_id uuid NOT NULL, -- The upstream process being depended on (alusprotsess)
    dependency_type character varying(50), -- e.g., 'DATA_FLOW', 'OPERATIONAL_TRIGGER'
    description text,
    created_at timestamp with time zone DEFAULT now(),
    CONSTRAINT pk_bp_dependencies PRIMARY KEY (id),
    CONSTRAINT fk_bp_dep_tenant FOREIGN KEY (tenant_id) REFERENCES public.app_tenants(id) ON DELETE CASCADE,
    CONSTRAINT fk_bp_dep_primary FOREIGN KEY (primary_process_id) REFERENCES public.business_processes(id) ON DELETE CASCADE,
    CONSTRAINT fk_bp_dep_target FOREIGN KEY (depends_on_process_id) REFERENCES public.business_processes(id) ON DELETE CASCADE,
    CONSTRAINT uq_bp_dependency UNIQUE (primary_process_id, depends_on_process_id)
);
```

CREATE INDEX ix_bp_dep_primary ON public.business_process_dependencies USING btree (primary_process_id);

3. Core Business Logic & SOPs (Service Layer)

You must implement the following three E-ITS rules in your Python/FastAPI service layer under the v2 modules:

SOP 1: Automatic Protection Need (Kaitsetarve) Calculation

- Rule: The overall protection need (kaitsetarbe koondtase) of a business process or asset must automatically match the highest value (GREATEST) among its Confidentiality (Konfidentsiaalsus), Integrity (Terviklus), and Availability (Käideldavus) dimensions.
- Values Allowed: NORMAL (Normaalne), HIGH (Suur), VERY_HIGH (Väga suur).
- Enforcement: Do not trust frontend calculations. Intercept inputs on POST/PUT in protection_need_summaries and execute the max-value allocation logic server-side.

SOP 2: Justification & Top-Management Approval (Põhjendus ja kinnitamine)

- Rule: E-ITS states that risk management and residual risks (jääkriskid) must be evaluated and accepted at the executive/CISO management level[cite: 1, 6].
- Enforcement: If protection_need calculates to HIGH or VERY_HIGH, the database record cannot be saved unless:
    1. The justification (kaitsetarbe määramise põhjendus) field is filled with text length >= 20 characters.
    2. The approved_by field matches a valid local_user_id holding an administrative/ISM role (infoturbejuht või tippjuhtkond).

SOP 3: Protection Need Inheritance & Asset Cascading (Kaitsetarbe pärandumine)

- Rule: When a business process protection need is set or upgraded, all supporting assets linked to it via process_assets must have their threat profiles reviewed.
- Enforcement: When a process is updated to HIGH or VERY_HIGH, automatically generate alerts (using the system's alert_level enum) targeting the ism role. The alert must flag the sub-assets (process_assets.asset_id) indicating that their baseline security level must be audited against standard or high-level measures (eits_catalog_measures.measure_level).

4. API v2 Endpoint Contracts

All routing and controller definitions for these newly implemented features must live strictly within backend/app/api/v2/.

GET /api/v2/business-processes/{id}/dependencies

- Description: Retrieves a flat list or tree structure of upstream and downstream process dependencies for cascading impact analysis[cite: 5].
- Security: Must strictly filter by the authenticated session's tenant_id[cite: 9]. Cross-tenant leakage must yield a 404 Not Found response code.

POST /api/v2/business-processes/{id}/dependencies

- Payload Schema:
- JSON

```
{
  "depends_on_process_id": "uuid",
  "dependency_type": "DATA_FLOW",
  "description": "String description of info sharing"
}
```

-
-

* **Validation:** Prevent self-referential dependencies and circular process loops (e.g., A depends on B, B depends on A).

---

5. Agent Verification Checklist

Before completing your code generation block, ensure you can answer "Yes" to all the following items:

1. Did you place the endpoints under `/api/v2/` instead of legacy `/api/v1/` routes?
2. Are Alembic database migrations written using clean constraints and foreign keys[cite: 9]?
3. Does tenant isolation utilize strict parameters using `tenant_id` context injection[cite: 9]?
4. Does the service automatically compute the highest C-I-A level as the primary process protection requirement[cite: 3]?